

C Programming Mastery Guide

A Systems-Level Study Companion (C99/C11)

This guide follows your syllabus (Sections IV–XIII). Every concept is treated with: **(1) Concept Breakdown**, **(2) Syntax & Standard Usage**, **(3) Common Pitfalls & Bugs**, **(4) Practical Code Example**, **(5) Hands-On Exercises**.

Pointers, Linked Lists, and File Management get extended memory-diagram treatment (stack vs heap) as requested.

Table of Contents

- IV. Basic Elements of C
 - V. Input/Output
 - VI. Control Element I (Selection)
 - VII. Control Element II (Loops)
 - VIII. Functions
 - IX. Arrays and Strings
 - X. Pointers (extended)
 - XI. User-Defined Types
 - XII. Linked Lists (extended)
 - XIII. File Management (extended)
-

IV. Basic Elements of C Language

IV.1 Header Files

1. Concept Breakdown

A header file (`.h`) contains declarations (function prototypes, macros, type definitions) that the preprocessor textually pastes into your `.c` file via `#include` before compilation even begins. The CPU never "sees" header files — they exist purely to let the *compiler* know what symbols mean before linking resolves their addresses. They exist to enable **separate compilation**: you can compile `.c` files independently and link the resulting object files, as long as each file agrees on the declared shape (signature) of shared functions/data.

- `#include <stdio.h>` — angle brackets: compiler searches system include paths.
- `#include "myheader.h"` — quotes: compiler searches local project directory first.

2. Syntax & Standard Usage

```
/* myheader.h */
#ifndef MYHEADER_H    /* include guard: prevents double inclusion */
```

```
#define MYHEADER_H

void greet(const char *name); /* declaration only, no body */

#endif
```

```
/* main.c */
#include <stdio.h> /* standard library header */
#include "myheader.h" /* your own header */

int main(void) {
    greet("Chu");
    return 0;
}
```

3. Common Pitfalls & Bugs

- Forgetting `#ifndef/#define/#endif` include guards → "redefinition" compile errors when a header is included twice (directly and transitively).
- Putting function **definitions** (with bodies) in headers included by multiple `.c` files → "multiple definition" linker errors. Use `static inline` or `extern` correctly if you must.
- Circular includes (A.h includes B.h, B.h includes A.h) → resolved by guards but can still cause incomplete-type errors.

4. Practical Code Example

```
/* shapes.h */
#ifndef SHAPES_H
#define SHAPES_H
double circle_area(double radius);
#endif

/* shapes.c */
#include "shapes.h"
#define PI 3.14159265
double circle_area(double radius) { return PI * radius * radius; }

/* main.c */
#include <stdio.h>
#include "shapes.h"
int main(void) {
    printf("Area: %.2f\n", circle_area(2.0));
    return 0;
}
/* compile: gcc main.c shapes.c -o app */
```

5. Hands-On Exercises

- *Basic*: Create `mathutils.h/.c` with a `int square(int)` function, call it from `main.c`.
 - *Challenging*: Split a single-file temperature-conversion program into 3 modules (`convert.h/.c`, `io.h/.c`, `main.c`) using proper include guards and compile them together with separate `gcc -c` steps followed by linking.
-

IV.2 Reserved Words (Keywords)

1. Concept Breakdown

Keywords are words reserved by the C standard with fixed syntactic meaning (`int`, `return`, `if`, `for`, `static`, `volatile`, etc.). The compiler's lexer treats them differently from identifiers — they cannot be redefined or used as variable names. There are 32 keywords in C89 and more added in C99 (`inline`, `restrict`, `_Bool`, `_Complex`) and C11 (`_Atomic`, `_Generic`, `_Noreturn`, `_Static_assert`, `_Thread_local`).

2. Syntax & Standard Usage

```
static int counter = 0;    /* 'static': keeps value between calls, file-  
scope linkage */  
const int MAX = 100;      /* 'const': read-only binding */  
volatile int flag;        /* 'volatile': tells compiler not to optimize  
away reads (e.g. hardware register) */
```

3. Common Pitfalls & Bugs

- Trying to name a variable `int class`; or `int new`; is fine in C (these are C++ keywords, not C keywords) — but using actual C keywords like `int int`; is a syntax error.
- Misunderstanding `static`: at file scope it means "internal linkage" (private to this .c file); inside a function it means "persists across calls." Same word, two different meanings.
- Forgetting `volatile` on memory-mapped hardware registers or signal-handler-shared variables → compiler optimizes away necessary reads (a classic embedded-systems bug relevant to your ESP32 fingerprint device work).

4. Practical Code Example

```
#include <stdio.h>  
int next_id(void) {  
    static int id = 0;    /* persists across calls, initialized once */  
    return ++id;  
}  
int main(void) {  
    for (int i = 0; i < 3; i++) printf("%d\n", next_id()); /* 1 2 3 */  
    return 0;  
}
```

5. Hands-On Exercises

- *Basic*: List and categorize 10 keywords into "type-related," "control-flow," and "storage-class."
 - *Challenging*: Write a function using `static` local storage to implement a simple call counter/rate limiter, and explain in comments the difference vs. making the variable global.
-

IV.3 Comments

1. Concept Breakdown

Comments are stripped entirely by the preprocessor/compiler before compilation — they have zero runtime cost and zero binary footprint. They exist purely for human communication (documentation, intent, TODOs).

2. Syntax & Standard Usage

```
// single-line comment (C99+)
/* multi-line
   comment block */
int x = 5; // trailing comment
```

3. Common Pitfalls & Bugs

- Nesting `/* ... /* ... */ ... */` is **not supported** — the first `*/` closes the comment, leaving trailing code uncommented and potentially breaking compilation.
- Accidentally commenting out a closing brace or semicolon while "temporarily disabling" code.
- Relying on comments instead of self-documenting code/asserts — comments rot and go stale as code changes.

4. Practical Code Example

```
#include <stdio.h>
/* Computes factorial iteratively.
   Precondition: n >= 0 */
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i; // multiply running product
    }
    return result;
}
int main(void) {
    printf("%d\n", factorial(5)); // 120
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Add proper header comments (purpose, params, return) to a function you've already written.
- *Challenging*: Write a small C program that intentionally demonstrates the "nested comment" trap, causing a real compile bug, then fix it.

IV.4 Identifiers

1. Concept Breakdown

Identifiers are names for variables, functions, types, and labels. Internally the compiler maps each identifier to a memory address or symbol-table entry (a location, offset, or register) — the name itself vanishes after compilation; only addresses remain in the binary (except in debug symbols).

2. Syntax & Standard Usage

```
int total_score;      /* valid: letters + underscore */
int _temp;            /* valid but leading underscore names are
reserved for library/system use – avoid */
int score2;           /* valid: digits allowed, not as first char */
```

Rules: letters, digits, underscore; cannot start with digit; case-sensitive; cannot be a keyword.

3. Common Pitfalls & Bugs

- Starting identifiers with underscore followed by uppercase (`_Foo`) or double underscore (`__foo`) — reserved for implementation/library use, can silently clash with compiler internals.
- Case-sensitivity bugs: `Total` and `total` are different identifiers — easy typo source.
- Overly similar names (`data1`, `data2`, `datat`) causing copy-paste bugs.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int student_count = 30;
    int studentCount = 31; /* legal but confusing – different variable!
*/
    printf("%d %d\n", student_count, studentCount);
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Identify which of these are valid identifiers: `2ndValue`, `_data`, `int`, `my-var`, `count_1`.
 - *Challenging*: Refactor a messy snippet with ambiguous names (`a`, `b`, `c`, `tmp2`) into descriptive, consistent identifiers following `snake_case`.
-

IV.5 Variables

1. Concept Breakdown

A variable is a named region of memory sized and interpreted according to its type. At runtime, memory is a flat array of bytes; a variable declaration tells the compiler "reserve N bytes here and treat reads/writes through this name as this type." Variables live in different memory regions based on **storage duration**: automatic (stack), static (data/bss segment), or dynamic (heap, via pointers).

2. Syntax & Standard Usage

```
int age = 25;           /* automatic (stack) variable, local to block */
static int total = 0;   /* static storage duration, persists whole
program */
extern int shared_flag; /* declared elsewhere, linked at build time */
```

3. Common Pitfalls & Bugs

- **Uninitialized variables:** `int x; printf("%d", x);` reads garbage (undefined behavior) — stack memory is not zeroed.
- **Shadowing:** declaring a local variable with the same name as an outer/global one, silently hiding it.
- **Scope confusion:** variables declared inside `{ }` blocks (e.g., inside an `if`) die at the closing brace.

4. Practical Code Example

```
#include <stdio.h>
int global_counter = 0; /* global: static storage duration */
void increment(void) {
    global_counter++;   /* modifies the single shared instance */
}
int main(void) {
    for (int i = 0; i < 5; i++) increment(); /* i is local to the loop */
    printf("Counter: %d\n", global_counter); /* 5 */
    return 0;
}
```

5. Hands-On Exercises

- *Basic:* Write a program declaring a local, a global, and a static-local variable; print all three across multiple function calls to observe lifetime differences.
- *Challenging:* Demonstrate and explain (via comments) an actual undefined-behavior bug caused by reading an uninitialized variable, then fix it with proper initialization.

IV.6 Literals / Constants

1. Concept Breakdown

Literals are fixed values written directly in source code (`42`, `3.14`, `'A'`, `"hi"`). The compiler embeds them directly into the instruction stream or a read-only data segment (`.rodata`) rather than allocating a "variable" for them, unless assigned to one.

2. Syntax & Standard Usage

```
int a = 42;           /* int literal */
long b = 42L;         /* long suffix */
unsigned c = 42U;     /* unsigned suffix */
double d = 3.14;      /* double literal (default for decimals) */
float e = 3.14f;      /* float literal, note suffix */
char f = 'A';         /* char literal (single quotes) */
const int MAX = 100;   /* named constant via const */
#define MAXSIZE 100    /* named constant via macro (no type, no memory) */
/*
```

3. Common Pitfalls & Bugs

- Integer overflow with literals too large for the assumed type (e.g., `int x = 9999999999`; overflows `int` silently on 32-bit `int` platforms).
- Forgetting `f` suffix causes unwanted implicit double → float narrowing conversions with warnings.
- `#define` macros have no type safety and no scope — can cause subtle bugs with operator precedence (`#define SQUARE(x) x*x` breaks on `SQUARE(1+2)`).

4. Practical Code Example

```
#include <stdio.h>
#define PI 3.14159
const int MAX_USERS = 500;
int main(void) {
    double area = PI * 2.0 * 2.0;
    printf("Area=%.2f, MaxUsers=%d\n", area, MAX_USERS);
    return 0;
}
```

5. Hands-On Exercises

- Basic*: Declare literals of every basic type (`int`, `float`, `char`, `string`) and print them with correct format specifiers.
- Challenging*: Reproduce the classic `#define SQUARE(x) x*x` macro-expansion bug with `SQUARE(1+2)`, observe the wrong output, then fix it using parentheses `((x)*(x))`.

IV.7 Separators

1. Concept Breakdown

Separators (`;`, `,`, `{ }`, `( )`, `[ ]`) are syntactic tokens that the parser uses to delimit statements, expressions, and scopes. They carry no runtime value — they exist to make the grammar of C unambiguous to the compiler's parser.

2. Syntax & Standard Usage

```
int a = 1, b = 2;           /* comma separates declarations */
int arr[10];               /* brackets: array declarator */
void func(int x, int y);   /* parens delimit parameter list */
{
    int local = 5;         /* braces delimit a block/scope */
}                          /* semicolon terminates statements */
```

3. Common Pitfalls & Bugs

- Missing semicolon → compile error, sometimes on the *next* line, confusing beginners.
- Stray semicolon after `if/for/while` condition creates an empty statement bug: `if (x > 0); { printf("positive\n"); }` — the block always runs regardless of condition!
- Comma operator vs comma separator confusion in `for` loops: `for (i=0, j=10; ...)` is valid (comma operator), but is different from declaration commas.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int x = 5;
    if (x > 0)          /* NOTE: no semicolon here – that's correct */
    {
        printf("Positive\n");
    }
    return 0;
}
```

PROF

5. Hands-On Exercises

- *Basic*: Write a program with 3 different separator types (`,`, `;`, `{ }`) used correctly and label each in a comment.
- *Challenging*: Intentionally reproduce the stray-semicolon-after-if bug, show the misleading output, then fix it and explain why the compiler didn't flag it as an error.

IV.8 Datatypes

1. Concept Breakdown

Datatypes tell the compiler how many bytes to reserve and how to interpret the bit pattern (signed integer, unsigned integer, IEEE-754 float, etc.). The CPU itself just moves and operates on raw bits; the

`type` is a compile-time contract about what those bits mean. Sizes are platform/ABI dependent (use `sizeof` — never assume).

2. Syntax & Standard Usage

```
#include <stdio.h>
#include <stdint.h> /* fixed-width types — preferred for portability,
esp. embedded (ESP32) work */

int main(void) {
    char c = 'A';          // usually 1 byte
    int i = 42;             // usually 4 bytes
    short s = 10;          // usually 2 bytes
    long l = 100000L;       // >=4 bytes
    long long ll = 10000000000LL;
    float f = 3.14f;        // 4 bytes, ~7 sig figs
    double d = 3.14159265358; // 8 bytes, ~15 sig figs
    unsigned int u = 4000000000U;
    uint8_t byte_val = 255; // exactly 8 bits, unsigned — great for
GPIO/registers
    int32_t exact32 = -100; // exactly 32 bits, signed

    printf("sizeof(int)=%zu sizeof(double)=%zu\n", sizeof(int),
sizeof(double));
    return 0;
}
```

3. Common Pitfalls & Bugs

- Assuming `int` is always 4 bytes — it's only guaranteed to be *at least* 16 bits by the standard.
- Signed/unsigned comparison bugs: `if (-1 < some_unsigned_var)` can evaluate false because `-1` gets implicitly converted to a huge unsigned number.
- Integer overflow (signed overflow is **undefined behavior** in C, not just wraparound).
- Float precision: `0.1 + 0.2 != 0.3` due to binary floating-point representation — never compare floats with `==`.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    unsigned int a = 3;
    unsigned int b = 5;
    int diff = a - b;          /* underflow trap if stored as unsigned
*/
    printf("diff as int: %d\n", diff);      /* -2, fine because cast to
int */
    unsigned int bad = a - b;
    printf("diff as unsigned: %u\n", bad); /* huge positive number! */
}
```

```
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Print `sizeof()` for every basic C type on your machine and compare to the standard's minimum guarantees.
- *Challenging*: Write a function that deliberately triggers signed integer overflow (e.g., `INT_MAX + 1`) using `<limits.h>`, observe the behavior with different compiler flags (`-fwrapv` vs default), and explain why relying on wraparound is unsafe.

IV.9 Format Specifiers

1. Concept Breakdown

Format specifiers (`%d`, `%f`, `%s`, etc.) tell `printf/scanf` how many bytes to read from the variadic argument list and how to interpret those bits. Because C's variadic functions have **no runtime type information**, a mismatched specifier reads/writes the wrong number of bytes — this is a real, exploitable bug class.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int i = 10; float f = 3.14f; double d = 3.14159;
    char c = 'X'; char *s = "hello";
    printf("%d %f %lf %c %s\n", i, (double)f, d, c, s);
    /* note: printf always promotes float args to double in varargs –
    use %f for both */
    printf("%5d|%-5d|%05d\n", i, i, i); /* width, left-align, zero-pad
    */
    printf("%x %o %p\n", 255, 8, (void*)&i); /* hex, octal, pointer */
    return 0;
}
```

PROF

3. Common Pitfalls & Bugs

- Mismatched specifier and argument type (`%d` for a `long`) is undefined behavior — on 64-bit systems this often corrupts subsequent output or crashes.
- Using `%f` in `scanf` for a `double` instead of `%lf` — `scanf` (unlike `printf`) does NOT auto-promote, so this writes only 4 bytes into an 8-byte variable, corrupting adjacent memory.
- Forgetting `&` when passing variables to `scanf`.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
```

```

double price;
printf("Enter price: ");
scanf("%lf", &price);          /* MUST use %lf, not %f, for double
*/
printf("Price entered: %.2f\n", price);
return 0;
}

```

5. Hands-On Exercises

- *Basic*: Print an `int`, `float`, `double`, and `char*` each with correct specifiers plus width/precision formatting.
- *Challenging*: Reproduce the `%f`-instead-of-`%lf` `scanf` bug with a `double`, observe corrupted values in a neighboring variable, and explain via a stack-memory diagram why adjacent memory got clobbered.

IV.10 Escape Sequences

1. Concept Breakdown

Escape sequences let you embed non-printable or special characters (newline, tab, null, backslash itself) inside character/string literals using a backslash. The compiler translates the two-character source sequence (`\n`) into a single byte (0x0A) in the compiled binary — it's a *compile-time* text substitution, not a runtime operation.

2. Syntax & Standard Usage

```

#include <stdio.h>
int main(void) {
    printf("Line1\nLine2\tTabbed\n");
    printf("Quote: \"C is fun\"\n");
    printf("Backslash: \\ and null char has code %d\n", '\0');
    printf("Bell:\a Carriage:\r\n");
    char newline = '\n';
    printf("Hex escape: \x41\n"); /* 'A' */
    return 0;
}

```

3. Common Pitfalls & Bugs

- Forgetting `\0` string terminator when manually building char arrays → string functions read past the buffer (undefined behavior, security risk).
- Using a single backslash in Windows paths ("`C:\new\test`") — `\n` and `\t` get misinterpreted as escapes rather than literal path characters. Must use `\\` or raw path handling.
- Confusing `'\0'` (null character, value 0) with `'0'` (character zero, value 48).

4. Practical Code Example

```

#include <stdio.h>
#include <string.h>
int main(void) {
    char buffer[6];
    strcpy(buffer, "Hi"); /* strcpy auto-adds '\0' */
    printf("[%s] length=%zu\n", buffer, strlen(buffer)); /* [Hi]
length=2 */
    buffer[2] = '!'; buffer[3] = '\0'; /* manually terminate after
modification */
    printf("[%s]\n", buffer);
    return 0;
}

```

5. Hands-On Exercises

- *Basic*: Print a small formatted "receipt" using `\t` and `\n` for alignment.
- *Challenging*: Manually construct a char array (not using `strcpy`) representing "CAT", forget the `\0` on purpose, pass it to `printf("%s")`, and observe/explain the garbage output that follows — then fix it.

IV.11 Operators

1. Concept Breakdown

Operators map directly (often 1:1) to CPU instructions: `+` → **ADD**, `&` → bitwise **AND**, `<<` → shift-left, etc. Understanding operators at the CPU level explains both their performance and their subtle behavioral edge cases (e.g., signed shift behavior, short-circuit evaluation skipping instructions entirely).

2. Syntax & Standard Usage

```

#include <stdio.h>
int main(void) {
    int a = 10, b = 3;
    printf("%d %d %d %d %d\n", a+b, a-b, a*b, a/b, a%b); /* arithmetic
*/
    printf("%d %d %d %d\n", a>b, a<b, a==b, a!=b);          /* relational
*/
    printf("%d %d\n", (a>0 && b>0), (a>0 || b<0));          /* logical,
short-circuit */
    printf("%d %d %d %d\n", a & b, a | b, a ^ b, ~a);        /* bitwise
*/
    printf("%d %d\n", a << 1, a >> 1);                        /* shift */
    a += 5; a *= 2;                                           /*
compound assignment */
    printf("%d\n", a);
    int x = (a > b) ? a : b;                                  /* ternary
*/
    printf("%d\n", x);
}

```

```
    return 0;
}
```

3. Common Pitfalls & Bugs

- `=` vs `==` typo in conditions: `if (a = 5)` compiles fine and is always true — classic bug.
- Operator precedence surprises: `a & b == c` evaluates as `a & (b == c)` because `==` binds tighter than `&`.
- Short-circuit side effects: `if (ptr != NULL && ptr->value > 0)` relies on short-circuiting to avoid a null-pointer dereference — reversing the order crashes.
- Undefined behavior: shifting by $\geq$ the width of the type (`1 << 32` on a 32-bit int), or shifting negative signed numbers.
- Post vs pre increment inside complex expressions (`a[i++] = i;`) — order of evaluation is unspecified/undefined for some cases, causing non-portable results.

4. Practical Code Example

```
#include <stdio.h>
#include <stdbool.h>
bool is_leap_year(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}
int main(void) {
    for (int y = 1996; y <= 2004; y += 4) {
        printf("%d leap? %s\n", y, is_leap_year(y) ? "yes" : "no");
    }
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Write a calculator that reads two ints and an operator character, then applies `+` `-` `*` `/` `%` correctly (including divide-by-zero guard).
- *Challenging*: Implement bit-flag management (e.g., simulate 8 GPIO pin states packed into one `uint8_t`) using `|`, `&`, `~`, and `^` to set, clear, toggle, and check individual bits — directly relevant to your ESP32 embedded work.

—
PROF

IV.12 Type Casting (Type Conversion)

1. Concept Breakdown

Type conversion reinterprets or transforms a bit pattern from one type's representation to another.

Implicit conversion (usual arithmetic conversions) happens automatically per C's promotion rules;

explicit casting (`type`)`expr` is programmer-directed and overrides the compiler's default, sometimes reinterpreting bits (pointer casts) rather than converting values (numeric casts).

2. Syntax & Standard Usage

```

#include <stdio.h>
int main(void) {
    int a = 7, b = 2;
    double result = (double)a / b;    /* explicit cast prevents integer
division truncation */
    printf("%.2f\n", result);        /* 3.50, not 3 */

    double pi = 3.99;
    int truncated = (int)pi;          /* explicit narrowing:
truncates toward zero -> 3 */
    printf("%d\n", truncated);

    long big = 1000000L;
    int narrowed = (int)big;          /* explicit, but can overflow
on smaller int types */

    void *generic = &a;
    int *typed = (int*)generic;       /* pointer cast: reinterpret,
not convert value */
    printf("%d\n", *typed);
    return 0;
}

```

3. Common Pitfalls & Bugs

- Integer division truncation bug: `int avg = total / count;` silently drops the decimal — must cast one operand to `double` *before* the division.
- Narrowing casts losing data silently (`(int)3000000000L` on a 32-bit int) — no warning by default in many cases.
- Casting a `float/double` to an integer type when the value is out of range is undefined behavior.
- Casting pointers between unrelated types and dereferencing violates strict aliasing rules — undefined behavior even if "it works" on your compiler today.

PROF

4. Practical Code Example

```

#include <stdio.h>
int main(void) {
    int scores[5] = {90, 85, 77, 92, 88};
    int sum = 0;
    for (int i = 0; i < 5; i++) sum += scores[i];
    double average = (double)sum / 5;    /* cast BEFORE division */
    printf("Average: %.2f\n", average);  /* 86.40 */
    return 0;
}

```

5. Hands-On Exercises

- *Basic*: Write a program computing the average of an int array, first without casting (showing the truncation bug), then fixed with an explicit cast.
- *Challenging*: Write a function that packs 4 separate `uint8_t` bytes into a single `uint32_t` using shifts (no casting library needed) and another that unpacks them back out — a common technique in embedded protocol/buffer parsing.

V. Input / Output

1. Concept Breakdown

C's standard I/O (`stdio.h`) is a thin, buffered wrapper around OS system calls (`read/write`). Each stream (`stdin`, `stdout`, `stderr`) is a `FILE*` structure holding a buffer, a file descriptor, and position state. `stdout` is normally *line-buffered* when attached to a terminal and *fully buffered* when redirected to a file — this is why output ordering can look different depending on where you run a program.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int age; char name[50];

    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);          /* SAFER than scanf("%s")
- bounds-checked */
    name[strcspn(name, "\n")] = '\0';          /* strip trailing newline
from fgets */

    printf("Enter your age: ");
    scanf("%d", &age);

    fprintf(stdout, "Hello %s, age %d\n", name, age);
    fprintf(stderr, "Debug: input processed\n"); /* stderr: unbuffered,
for diagnostics */
    return 0;
}
```

3. Common Pitfalls & Bugs

- `scanf("%s", buf)` has **no bounds checking** — classic buffer overflow vector. Prefer `fgets` + parsing, or `scanf("%49s", buf)` with an explicit width limit.
- Leftover `\n` in the input buffer after `scanf("%d", ...)` gets silently consumed by a subsequent `fgets`, causing it to read an empty line — a very common "why did my input get skipped" bug.
- Not checking `scanf`'s return value (number of successfully matched items) — silently proceeding with garbage/uninitialized variables on bad input.
- Mixing buffered `stdout` writes with unbuffered `stderr` writes can appear out of order when redirected — use `fflush(stdout)` when interleaving matters.

4. Practical Code Example

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char line[100];
    int total = 0, count = 0;
    printf("Enter numbers, one per line, 'q' to quit:\n");
    while (fgets(line, sizeof(line), stdin)) {
        if (line[0] == 'q') break;
        int value;
        if (sscanf(line, "%d", &value) == 1) { /* always validate parse
success */
            total += value; count++;
        } else {
            fprintf(stderr, "Invalid input ignored: %s", line);
        }
    }
    if (count > 0) printf("Average: %.2f\n", (double)total / count);
    return 0;
}
```

5. Hands-On Exercises

- *Basic:* Write a program that safely reads a name and age using `fgets/sscanf` (not raw `scanf("%s")`), and prints a greeting.
- *Challenging:* Build a tiny REPL that reads lines from `stdin`, supports commands `add <n>`, `sum`, and `quit`, validating every input line, gracefully rejecting malformed commands without crashing.

VI. Control Element I (Selection / Branching)

PROF

VI.1 Statements and Blocks

1. Concept Breakdown

A *statement* is the smallest executable unit (ends in `;` or is a compound block). A *block* `{ ... }` groups statements and defines a new lexical scope — variables declared inside die when the block ends, and the compiler generates instructions to allocate/deallocate their stack space at block entry/exit.

2. Syntax & Standard Usage

```
int main(void) {
    int x = 10;           /* statement */
    {                     /* block start: new scope */
        int y = 20;       /* y only exists in this block */
        x = x + y;
    }                     /* y is destroyed here */
}
```



```
    return 0;
}
```

3. Common Pitfalls & Bugs

- Believing a single statement after `if/for` without braces is part of a "block" — only the *next single statement* is controlled; adding a second line unintentionally moves it outside the conditional.
- Variable shadowing across nested blocks causing confusing bugs.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int total = 0;
    for (int i = 1; i <= 3; i++) {
        int square = i * i; /* scoped to loop body */
        total += square;
    }
    printf("Total: %d\n", total);
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Write nested blocks demonstrating variable shadowing; print values at each scope level.
- *Challenging*: Find and fix a real bug caused by a missing brace (write the buggy version first, showing the dangling-else-style effect, then correct it).

VI.2 If-Else

1. Concept Breakdown

PROF

`if/else` compiles to a conditional branch instruction (e.g., `JNE/JE` on x86, `BEQ/BNE` on ARM) — the CPU evaluates the condition into a flag register and jumps over the "else" block or into it accordingly.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int score = 72;
    if (score >= 90) {
        printf("A\n");
    } else if (score >= 70) {
        printf("B\n");
    } else {
        printf("C or below\n");
    }
}
```

```
    return 0;
}
```

3. Common Pitfalls & Bugs

- The classic **dangling else**: nested `if` without braces attaches `else` to the nearest unmatched `if`, not necessarily the one you intended.
- Comparing floating-point equality directly in conditions (`if (x == 0.3)`) — precision errors cause false negatives.
- Assignment instead of comparison (`if (flag = 0)`) — always true or always false unintentionally.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int temp_c = 38;
    if (temp_c > 37) {
        printf("Fever detected: %d C\n", temp_c);
    } else {
        printf("Normal temperature: %d C\n", temp_c);
    }
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Grade classifier that maps a numeric score to a letter grade using if-else chains.
- *Challenging*: Reproduce a dangling-else bug on purpose with nested unbraced `ifs`, show the wrong branch executing, then fix it with explicit braces.

VI.3 Else-If

1. Concept Breakdown

`else if` is not a separate keyword — it's just `else { if (...) ... }` written without the extra braces, chained. The compiler generates a sequence of comparisons, each only reached if all previous ones failed (like a decision ladder) — this is $O(n)$ in the worst case, unlike `switch` which the compiler can sometimes optimize into a jump table.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int hour = 14;
    if (hour < 12) printf("Morning\n");
    else if (hour < 17) printf("Afternoon\n");
}
```

```
    else if (hour < 21) printf("Evening\n");
    else printf("Night\n");
    return 0;
}
```

3. Common Pitfalls & Bugs

- Ordering conditions incorrectly (broad condition checked before narrow one) means the narrow branch is never reached — e.g., checking `score >= 0` before `score >= 90`.
- Not including a final `else` to catch unexpected/invalid values.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int bmi_category_input = 27;
    if (bmi_category_input < 18) printf("Underweight\n");
    else if (bmi_category_input < 25) printf("Normal\n");
    else if (bmi_category_input < 30) printf("Overweight\n");
    else printf("Obese\n");
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Convert a numeric month (1-12) to a season name using else-if chains.
- *Challenging*: Write a tax-bracket calculator with 5 progressive brackets, ensuring correct ordering and boundary handling (test edge values exactly at bracket boundaries).

VI.4 Switch Case

1. Concept Breakdown

`switch` on integral types often compiles to a **jump table** when cases are dense (an array of addresses indexed directly by the case value — $O(1)$ dispatch), or a comparison chain when sparse. This is why `switch` can only operate on integer/char/enum constant expressions, not floats or strings.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int day = 3;
    switch (day) {
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        default: printf("Unknown\n"); break;
    }
}
```

```
    return 0;
}
```

3. Common Pitfalls & Bugs

- **Missing `break`:** execution "falls through" into the next case — sometimes intentional (grouping cases) but often a serious bug.
- Forgetting **`default`**: — silently ignoring unexpected values.
- Using **`switch`** on a **`double/float/string`** — not allowed in C at all (unlike some other languages).
- Declaring a variable inside a **`case`** without its own block **`{ }`** can cause "jump bypasses initialization" errors if a later **`case`** label appears before the variable's scope naturally ends.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    char grade = 'B';
    switch (grade) {
        case 'A':
        case 'B': /* intentional fallthrough:
grouping */
            printf("Good job!\n");
            break;
        case 'C':
            printf("Average\n");
            break;
        default:
            printf("Needs improvement\n");
    }
    return 0;
}
```

PROF

5. Hands-On Exercises

- *Basic:* Simple menu-driven calculator using **`switch`** on an operator char (+ - * /).
- *Challenging:* Intentionally omit a **`break`** to create a fallthrough bug in a multi-case menu program, demonstrate the wrong cascading output, then fix it and add a comment **`/* fallthrough intentional */`** style marker where fallthrough IS wanted elsewhere in the same switch.

VII. Control Element II (Repetition / Loops)

VII.1 While Loop

1. Concept Breakdown

`while` compiles to: evaluate condition → conditional branch to exit → loop body → unconditional jump

back to condition check. It's the most primitive loop — entry-controlled, condition checked *before* every iteration including the first, so the body may run zero times.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int i = 0;
    while (i < 5) {
        printf("%d ", i);
        i++;
    }
    printf("\n");
    return 0;
}
```

3. Common Pitfalls & Bugs

- Infinite loop from forgetting to update the loop-control variable.
- Off-by-one errors in the condition ($\leq$ vs $<$) causing one extra or one missing iteration.
- Modifying the loop variable unexpectedly inside the body, causing skipped or repeated iterations.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int n, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    while (n > 0) {
        sum += n % 10;    /* add last digit */
        n /= 10;         /* remove last digit */
    }
    printf("Digit sum: %d\n", sum);
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Print numbers 1 to 10 using `while`.
- *Challenging*: Implement a number-guessing game where the loop continues until the user guesses correctly or runs out of a fixed number of attempts, tracking attempt count.

VII.2 Do-While

1. Concept Breakdown

`do-while` is exit-controlled: the body executes first, *then* the condition is checked. At the machine level,

it's actually the most "natural" loop for a CPU (jump-back-if-true is a single branch instruction), whereas `while/for` need an extra branch to skip the body on first entry — compilers often transform `while` loops into `do-while` form internally when they can prove the body runs at least once.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    int choice;
    do {
        printf("Menu: 1-Add 2-Remove 3-Exit\nChoice: ");
        scanf("%d", &choice);
    } while (choice != 3);
    printf("Goodbye\n");
    return 0;
}
```

3. Common Pitfalls & Bugs

- Forgetting the trailing semicolon after `while(condition);` — required for `do-while`, unlike `while` and `for`.
- Using `do-while` when a zero-iteration case should be possible (input validation menus are the right use-case, but data processing over possibly-empty collections often isn't).

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int password;
    int attempts = 0;
    do {
        printf("Enter PIN: ");
        scanf("%d", &password);
        attempts++;
    } while (password != 1234 && attempts < 3);

    if (password == 1234) printf("Access granted\n");
    else printf("Access denied after %d attempts\n", attempts);
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Build an input-validation loop that keeps asking for a number between 1-100 until valid.
- *Challenging*: Implement a simple ATM-style PIN entry with a maximum of 3 attempts and account lockout state, using `do-while`.

VII.3 For Loop

1. Concept Breakdown

`for (init; cond; update)` is syntactic sugar around `while`, bundling the three loop-control pieces into one line. The `init` runs once; the compiler still generates essentially the same branch/jump structure as an equivalent `while` loop — the value of `for` is purely organizational, keeping loop-control logic visually together and scoped to the loop.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    for (int i = 0; i < 5; i++) {
        printf("%d ", i);
    }
    printf("\n");

    /* multiple variables in a for loop using comma operator */
    for (int i = 0, j = 10; i < j; i++, j--) {
        printf("(%d,%d) ", i, j);
    }
    printf("\n");
    return 0;
}
```

3. Common Pitfalls & Bugs

- Classic **off-by-one**: `for (i = 0; i <= n; i++)` on an array of size `n` reads/writes one element past the end — buffer overrun.
- Modifying the loop counter inside the body in addition to the `update` clause, causing skipped/duplicated iterations.
- Using `float`/`double` as a loop counter — accumulated rounding error can cause the loop to run one time too many/few, or never terminate exactly at the boundary.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    int sum = 0;
    for (int i = 0; i < 5; i++) { /* correct bound: strictly less than
size */
        sum += arr[i];
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Print the multiplication table of a number 1-10 using **for**.
- *Challenging*: Deliberately write an off-by-one buffer overrun using **<=** on an array loop, compile with **-fsanitize=address**, observe the sanitizer's crash report, then fix the bound.

VII.4 Break and Continue

1. Concept Breakdown

break compiles to an unconditional jump to the instruction immediately after the loop/switch.

continue compiles to an unconditional jump to the loop's *update* step (for **for**) or condition check (for **while**/**do-while**) — skipping the remainder of the current iteration's body only.

2. Syntax & Standard Usage

```
#include <stdio.h>
int main(void) {
    for (int i = 1; i <= 10; i++) {
        if (i == 7) break;           /* exit loop entirely */
        if (i % 2 == 0) continue;   /* skip even numbers, go to next
iteration */
        printf("%d ", i);
    }
    printf("\n"); /* prints: 1 3 5 */
    return 0;
}
```

3. Common Pitfalls & Bugs

- **break** inside a nested loop only exits the **innermost** loop, not all levels — a common source of "why is it still looping" confusion. (Use a flag variable or **goto** for multi-level breaks.)
- **continue** inside a **for** loop still executes the update clause (**i++**), but inside a **while** loop, if the increment is *after* the **continue** point in the body, it gets skipped — leading to accidental infinite loops.

4. Practical Code Example

```
#include <stdio.h>
int main(void) {
    int found = 0;
    int matrix[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    int target = 5;
    for (int i = 0; i < 3 && !found; i++) {
        for (int j = 0; j < 3; j++) {
            if (matrix[i][j] == target) {
                printf("Found at [%d][%d]\n", i, j);
                found = 1;
            }
        }
    }
}
```



```

        break;                /* only exits inner loop; outer flag
stops outer */
    }
}
}
return 0;
}

```

5. Hands-On Exercises

- *Basic:* Print numbers 1-20, skipping multiples of 3 using `continue`, and stop entirely once you reach 15 using `break`.
- *Challenging:* Write a prime-checker that uses `break` to exit its trial-division loop early upon finding a factor, and measure/compare (with comments) the number of iterations saved vs. a version without early exit.

VII.5 Goto and Labels

1. Concept Breakdown

`goto label;` compiles to a direct unconditional jump instruction to the labeled address — it is the rawest possible control-flow construct, mirroring what all higher-level control flow (`if`, loops) gets compiled *down to* at the assembly level. Its main legitimate modern use in C is centralized cleanup/error-handling in resource-heavy functions (a pattern common in the Linux kernel and embedded C).

2. Syntax & Standard Usage

```

#include <stdio.h>
int main(void) {
    int i = 0;
loop_start:                /* label */
    if (i >= 5) goto loop_end;
    printf("%d ", i);
    i++;
    goto loop_start;
loop_end:
    printf("\ndone\n");
    return 0;
}

```

3. Common Pitfalls & Bugs

- Overuse creates "spaghetti code" — jumps that criss-cross functions become nearly impossible to trace/debug.
- Jumping into the middle of a block can skip variable initialization, leaving variables in an undefined state when later code assumes they're initialized.
- Jumping out of nested loops is a legitimate, often-cited valid use, but many teams ban `goto` entirely by style-guide regardless.

4. Practical Code Example (idiomatic cleanup pattern)

```
#include <stdio.h>
#include <stdlib.h>
int process(void) {
    int *buf1 = malloc(100);
    if (!buf1) goto fail;
    int *buf2 = malloc(200);
    if (!buf2) goto fail_buf1;

    /* ... use buf1, buf2 ... */
    printf("Processing succeeded\n");
    free(buf2);
    free(buf1);
    return 0;

fail_buf1:
    free(buf1);
fail:
    fprintf(stderr, "Allocation failed\n");
    return -1;
}
int main(void) { return process(); }
```

5. Hands-On Exercises

- *Basic:* Use `goto` to break out of a triple-nested loop searching a 3D-style structure (three nested `for`s) the moment a target value is found.
- *Challenging:* Rewrite the "resource cleanup" example above with 4 sequential allocations, ensuring every failure path frees exactly the resources already successfully allocated (no leaks, no double-frees) — verify with `valgrind` or `-fsanitize=address`.

VIII. Functions

1. Concept Breakdown

A function is a named, reusable block of code with its own **stack frame** (activation record). When called, the CPU pushes a return address (and, per calling convention, arguments/registers) onto the stack, jumps to the function's code, executes with its own local variables allocated in that frame, then pops the frame and jumps back on `return`. This is why local variables vanish after the function returns, and why deep/unbounded recursion exhausts the stack (**stack overflow**).

2. Syntax & Standard Usage

```
#include <stdio.h>

/* prototype (declaration) – lets callers appearing earlier in the file
```

```

know the signature */
int add(int a, int b);

int main(void) {
    int result = add(3, 4);
    printf("%d\n", result);
    return 0;
}

/* definition */
int add(int a, int b) {
    return a + b;    /* value copied back to caller – "return by value"
*/
}

/* pass-by-reference emulation via pointer, since C has no true
reference params */
void increment(int *value) {
    (*value)++;
}

```

3. Common Pitfalls & Bugs

- Forgetting that C passes arguments **by value** by default — modifying a parameter inside the function does not affect the caller's variable unless you pass a pointer.
- Returning a pointer to a local (stack) variable — the memory is invalid the instant the function returns (**dangling pointer**), yet the pointer value itself still "looks" valid, causing intermittent bugs.
- Missing/mismatched prototypes causing implicit int assumptions pre-C99, or outright compile errors in C99/C11 (implicit function declarations were removed).
- Unbounded/incorrect-base-case recursion causing stack overflow.

4. Practical Code Example

```

#include <stdio.h>

/* BAD: returns dangling pointer to stack memory - shown for teaching,
do not use */
/*
int* bad_function(void) {
    int local = 42;
    return &local;    // undefined behavior: local dies when function
returns
}
*/

/* GOOD: caller owns the memory, function fills it in */
void compute_stats(const int arr[], int n, int *out_min, int *out_max,
double *out_avg) {
    int sum = 0;
    *out_min = arr[0];

```

```

    *out_max = arr[0];
    for (int i = 0; i < n; i++) {
        sum += arr[i];
        if (arr[i] < *out_min) *out_min = arr[i];
        if (arr[i] > *out_max) *out_max = arr[i];
    }
    *out_avg = (double)sum / n;
}

int main(void) {
    int scores[] = {88, 92, 79, 95, 84};
    int mn, mx; double avg;
    compute_stats(scores, 5, &mn, &mx, &avg);
    printf("min=%d max=%d avg=%.2f\n", mn, mx, avg);
    return 0;
}

```

5. Hands-On Exercises

- *Basic:* Write a `swap(int *a, int *b)` function using pointers, and a `main` that demonstrates it actually swaps the caller's variables.
- *Challenging:* Implement recursive `int fibonacci(int n)` AND an iterative version; measure/compare call counts (add a static counter) for `n=30`, and explain in comments why naive recursive Fibonacci is exponential time while the iterative version is linear.

IX. Arrays and Strings

1. Concept Breakdown

An array is a single contiguous block of memory holding `n` elements of the same type — `arr[i]` is really computed as `*(arr + i * sizeof(type))`; there's no bounds checking at the language level, only whatever you enforce yourself. A C "string" is simply a `char` array with a `'\0'` sentinel marking its logical end — string functions (`strlen`, `strcpy`, etc.) scan byte-by-byte until they hit that null terminator, which is why a missing terminator causes them to read arbitrarily far past the buffer.

PROF

Memory layout (stack array of 5 ints):

```

Address:  1000  1004  1008  1012  1016
Value:    [10] [20] [30] [40] [50]
          arr[0] arr[1] arr[2] arr[3] arr[4]
arr decays to a pointer to arr[0] (address 1000) in most expressions.

```

2. Syntax & Standard Usage

```

#include <stdio.h>
#include <string.h>

```

```

int main(void) {
    int nums[5] = {1, 2, 3, 4, 5};
    int matrix[2][3] = {{1,2,3},{4,5,6}}; /* 2D array: row-major layout
in memory */

    char greeting[20] = "Hello";          /* char array = string, auto
null-terminated */
    char *name = "World";                 /* string literal: read-
only, stored in .rodata */

    strcat(greeting, ", ");               /* append — DANGEROUS if
buffer too small */
    strncat(greeting, name, sizeof(greeting) - strlen(greeting) - 1); /*
safer, bounded */
    printf("%s (%zu chars, buffer size %zu)\n", greeting,
strlen(greeting), sizeof(greeting));

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            printf("%d ", matrix[i][j]);
    printf("\n");
    return 0;
}

```

3. Common Pitfalls & Bugs

- **Buffer overflow:** writing past array bounds (`arr[5]` on a 5-element array) is undefined behavior — may silently corrupt adjacent stack memory (other variables, saved registers, or the return address itself) — a classic security vulnerability class).
- Using `strcpy/strcat/gets` without bounds checking — always prefer `strncpy/snprintf/fgets` in real code.
- Forgetting arrays don't carry their size at runtime — `sizeof(arr)` only works correctly in the scope where `arr` is still a real array, not after it has decayed to a pointer (e.g., inside a function that received it as a parameter, `sizeof(arr)` gives the pointer's size, not the array's).
- Comparing strings with `==` (compares pointers, not contents) instead of `strcmp`.
- Off-by-one in string buffer sizing: forgetting the `+1` byte needed for `'\0'`.

4. Practical Code Example

```

#include <stdio.h>
#include <string.h>

#define MAX_NAME 30

typedef struct { char name[MAX_NAME]; int score; } Student;

void print_report(const Student students[], int count) {
    for (int i = 0; i < count; i++) {

```

```

        printf("%-15s : %d\n", students[i].name, students[i].score);
    }
}

int main(void) {
    Student roster[3];
    strncpy(roster[0].name, "Alice", MAX_NAME - 1);
    roster[0].name[MAX_NAME-1] = '\0'; roster[0].score = 92;
    strncpy(roster[1].name, "Bob", MAX_NAME - 1);
    roster[1].name[MAX_NAME-1] = '\0'; roster[1].score = 85;
    strncpy(roster[2].name, "Carol", MAX_NAME - 1);
    roster[2].name[MAX_NAME-1] = '\0'; roster[2].score = 78;

    print_report(roster, 3);
    return 0;
}

```

5. Hands-On Exercises

- *Basic:* Write a function `int find_max(int arr[], int n)` and test it on several arrays including negative numbers.
- *Challenging:* Implement your own `my_strncpy(char *dest, const char *src, size_t dest_size)` that behaves like a *safe*, bounds-respecting `strncpy` — it must never write past `dest_size` bytes and must always null-terminate. Test it deliberately with a source string longer than the destination buffer and confirm no overflow occurs (verify with `-fsanitize=address`).

X. Pointers in C (*extended treatment*)

1. Concept Breakdown

A pointer is a variable whose value **is a memory address**. The CPU doesn't know or care what's "at" an address — a pointer's *type* is purely a compile-time annotation telling the compiler how many bytes to read/write and how to interpret them when you dereference. Every pointer itself also occupies memory (typically 4 or 8 bytes depending on architecture) — a pointer is a variable that stores the *location* of another variable.

Stack vs Heap — the core distinction:

STACK (automatic storage)

- Grows/shrinks automatically
- `malloc()/free()`
- Fast (just moves a stack pointer bookkeeping)
- Fixed size decided at compile time
- Freed automatically when scope ends
- Overflow = stack overflow (crash) time

HEAP (dynamic storage)

- Explicitly managed:
- Slower (allocator)
- Size decided at RUNTIME
- YOU must `free()` or it leaks
- Fragmentation possible over time

main()'s frame:	Heap (grows via malloc):
+-----+ <- higher addr	+-----+
return addr	[42][?][?][?] <- 4
ints, malloc'd	
saved regs	address: 0x9000
int x = 10 <- ptr here	+-----+
int *p -----> points to heap block above	
+-----+ <- lower addr, stack grows downward	

2. Syntax & Standard Usage

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* --- Basic pointer to a stack variable --- */
    int x = 10;
    int *p = &x;          /* p holds the ADDRESS of x */
    printf("x=%d, *p=%d, address=%p\n", x, *p, (void*)p);
    *p = 20;              /* dereference: modify x THROUGH the pointer */
    printf("x is now %d\n", x);

    /* --- Pointer arithmetic --- */
    int arr[3] = {100, 200, 300};
    int *ap = arr;         /* array decays to pointer to arr[0] */
    printf("%d %d %d\n", *ap, *(ap+1), *(ap+2)); /* pointer arithmetic
    == indexing */

    /* --- Dynamic (heap) allocation --- */
    int *heap_arr = malloc(5 * sizeof(int)); /* request 5 ints on the
    HEAP */
    if (heap_arr == NULL) { /* ALWAYS check
    malloc's return */
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }
    for (int i = 0; i < 5; i++) heap_arr[i] = i * i;
    for (int i = 0; i < 5; i++) printf("%d ", heap_arr[i]);
    printf("\n");
    free(heap_arr);        /* MUST free what you malloc – heap memory
    does not auto-clean */
    heap_arr = NULL;       /* good practice: avoid an accidental
    dangling-pointer reuse */

    /* --- Pointer to pointer --- */
    int y = 99;
    int *py = &y;
    int **ppy = &py;      /* pointer to a pointer */
    printf("y via **ppy = %d\n", **ppy);
}
```

```
    return 0;
}
```

3. Common Pitfalls & Bugs (with memory diagrams)

a) Dangling pointer — pointer outlives what it points to:

```
int *danger(void) {
    int local = 5;
    return &local;    /* BUG: &local becomes invalid the moment this
function returns */
}
/* Stack frame for danger() is POPPED after return — that memory may be
overwritten by the very next function call's frame. The returned
pointer still "looks" valid (it's a number) but reading through it
is undefined behavior. */
```

b) Memory leak — heap memory never freed:

```
void leaky(void) {
    int *p = malloc(100 * sizeof(int));
    /* ... used p ... but forgot free(p); */
} /* p (the pointer variable) is destroyed here, but the 100-int HEAP
BLOCK
    it pointed to is now unreachable and permanently lost until the
process exits. */
```

c) Double free / use-after-free:

```
int *p = malloc(sizeof(int));
free(p);
free(p);    /* BUG: double free — heap corruption, undefined behavior
*/
*p = 5;     /* BUG: use-after-free — writing into memory the allocator
may have reused */
```

d) NULL pointer dereference:

```
int *p = NULL;
*p = 5;     /* BUG: segmentation fault — dereferencing address 0 */
```

e) Wild/uninitialized pointer:


```
int *p;          /* uninitialized – contains GARBAGE, not NULL */
*p = 5;          /* BUG: writes to a random, unpredictable address */
```

4. Practical Code Example — dynamic array that grows (a mini realloc-based vector):

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    size_t size;
    size_t capacity;
} IntVector;

void vec_init(IntVector *v) {
    v->capacity = 4;
    v->size = 0;
    v->data = malloc(v->capacity * sizeof(int)); /* initial heap block
*/
}

void vec_push(IntVector *v, int value) {
    if (v->size == v->capacity) {
        v->capacity *= 2;
        int *new_data = realloc(v->data, v->capacity * sizeof(int)); /*
grow heap block */
        if (!new_data) { fprintf(stderr, "realloc failed\n"); exit(1); }
        v->data = new_data; /* realloc may have MOVED the block –
always reassign */
    }
    v->data[v->size++] = value;
}

void vec_free(IntVector *v) {
    free(v->data);
    v->data = NULL;
    v->size = v->capacity = 0;
}

int main(void) {
    IntVector v;
    vec_init(&v);
    for (int i = 1; i <= 10; i++) vec_push(&v, i * i);
    for (size_t i = 0; i < v.size; i++) printf("%d ", v.data[i]);
    printf("\n(size=%zu, capacity=%zu)\n", v.size, v.capacity);
    vec_free(&v); /* release heap memory back to the OS/allocator
*/
    return 0;
}
```

5. Hands-On Exercises

- *Basic*: Write a `void swap_via_pointers(int *a, int *b)` and confirm with `main` it swaps two independent stack variables. Draw (in comments) the addresses before/after.
 - *Challenging*: Extend the `IntVector` example into a `vec_pop`, `vec_get(index)` with bounds checking that returns an error code instead of crashing on out-of-range access, and a `vec_shrink_to_fit` that reallocates down to exactly `size` elements. Run the whole program under `valgrind --leak-check=full` (or `gcc -fsanitize=address,leak`) and confirm zero leaks/errors.
-

XI. User-Defined Datatypes and Variables (Structs, Unions, Enums, Typedef)

1. Concept Breakdown

- **struct**: groups different-typed fields into one contiguous memory block; each member is laid out at a specific (often padded, for alignment) offset from the struct's base address. Size = sum of members + padding for alignment.
- **union**: all members **share the same memory** (same starting address), sized to the largest member — writing one member overwrites the others' bytes. Used for memory-efficient variant types or byte-level reinterpretation.
- **enum**: named integer constants, improving readability over "magic numbers"; internally just `ints`.
- **typedef**: creates an *alias* for an existing type name — a purely compile-time convenience, zero runtime cost.

2. Syntax & Standard Usage

```
#include <stdio.h>
#include <string.h>

/* struct: distinct memory for each field */
typedef struct {
    char name[20];
    int age;
    float gpa;
} Student; /* typedef gives us the alias "Student"
instead of "struct Student" */

/* enum: named constants */
typedef enum { MON, TUE, WED, THU, FRI, SAT, SUN } Weekday;

/* union: overlapping memory */
typedef union {
    int i;
    float f;
    unsigned char bytes[4];
```

```

} Variant;

int main(void) {
    Student s = {"Chu", 22, 3.8f};
    printf("%s, %d, %.1f\n", s.name, s.age, s.gpa);

    Weekday today = WED;
    printf("Today enum value: %d\n", today);    /* prints 2 */

    Variant v;
    v.i = 65;                                  /* stores 65 into
shared memory */
    printf("as int: %d, first byte as char: %d\n", v.i, v.bytes[0]);
    /* v.f is now GARBAGE from int's perspective if read - union members
overlap! */

    return 0;
}

```

3. Common Pitfalls & Bugs

- **Struct padding surprises:** `sizeof(struct)` is often *larger* than the naive sum of member sizes due to alignment padding inserted by the compiler — never assume tight packing; use `sizeof()` explicitly rather than hardcoding sizes (critical for your ESP32 BOM/GPIO/register-mapping work where struct layout may need `#pragma pack` or `__attribute__((packed))` to match hardware register layouts exactly).
- **Union type confusion:** writing one member and reading a *different* member is technically undefined behavior outside of specific "type punning through a union" carve-outs — use with care, especially across compilers.
- Comparing structs with `==` is **not allowed** in C — must compare field-by-field or use `memcmp` (careful: `memcmp` also compares padding bytes, which may be uninitialized garbage).
- `enum` values are just `ints` with no compiler-enforced range checking — assigning an out-of-range integer to an enum variable compiles silently.
- Forgetting `typedef` still requires `struct Name var;` (without typedef) vs just `Name var;` (with typedef) — mixing the two conventions inconsistently across a codebase.

4. Practical Code Example

```

#include <stdio.h>

typedef enum { PENDING, IN_PROGRESS, DONE } Status;

typedef struct {
    int id;
    char title[40];
    Status status;
} Task;

const char* status_to_string(Status s) {

```

```

switch (s) {
    case PENDING:      return "Pending";
    case IN_PROGRESS:  return "In Progress";
    case DONE:         return "Done";
    default:           return "Unknown";
}

}

int main(void) {
    Task tasks[2] = {
        {1, "Design SRS", DONE},
        {2, "Build BOM", IN_PROGRESS}
    };
    for (int i = 0; i < 2; i++) {
        printf("[%d] %-15s : %s\n", tasks[i].id, tasks[i].title,
status_to_string(tasks[i].status));
    }
    printf("sizeof(Task) = %zu bytes (includes alignment padding)\n",
sizeof(Task));
    return 0;
}

```

5. Hands-On Exercises

- *Basic:* Define a `struct Point { int x, y; }`, write a function `double distance(struct Point a, struct Point b)`, and test with several point pairs.
- *Challenging:* Model a hardware register byte-packing scenario relevant to embedded work: define a `union SensorReading { float temperature; uint8_t raw_bytes[4]; }`, write a function that fills `raw_bytes` (simulating 4 bytes received over a UART/I2C line from your ESP32 fingerprint device) and then reads back `.temperature` — explain in comments the endianness assumption this relies on and why it's not portable across all architectures.

XII. Linked Lists (*extended treatment*)

1. Concept Breakdown

A linked list is a chain of dynamically-allocated **nodes**, each holding data plus a pointer to the next node. Unlike an array, elements are **not contiguous in memory** — each node can live anywhere on the heap; connectivity exists only through pointers. This trades $O(1)$ random access (arrays) for $O(1)$ insertion/deletion at arbitrary positions (no shifting required) and no need to know the size in advance.

Memory diagram — building a 3-node singly linked list:

STACK	HEAP
+-----+	+-----+-----+ +-----+-----+
+-----+-----+	
head (ptr) ----->	data=10 next •----> data=20 next •---
--> data=30 next=NULL	

+-----+-----+	+-----+-----+	+-----+-----+
+-----+-----+		
	addr 0x1000	addr 0x1050
addr 0x1090		

(order in memory is NOT sequential/contiguous – could be anywhere)

2. Syntax & Standard Usage

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next; /* self-referential struct: must use "struct
Node", typedef alias not yet complete */
} Node;

Node* create_node(int value) {
    Node *n = malloc(sizeof(Node)); /* heap allocation for the new
node */
    if (!n) { fprintf(stderr, "malloc failed\n"); exit(1); }
    n->data = value;
    n->next = NULL;
    return n;
}

void push_front(Node **head, int value) {
    Node *n = create_node(value);
    n->next = *head; /* new node points to old head */
    *head = n; /* head pointer updated via double-pointer
to affect caller's variable */
}

void print_list(Node *head) {
    for (Node *cur = head; cur != NULL; cur = cur->next) {
        printf("%d -> ", cur->data);
    }
    printf("NULL\n");
}

void free_list(Node *head) {
    Node *cur = head;
    while (cur != NULL) {
        Node *next = cur->next; /* save next BEFORE freeing current */
        free(cur);
        cur = next;
    }
}

int main(void) {
```

```

Node *head = NULL;
push_front(&head, 30);
push_front(&head, 20);
push_front(&head, 10);
print_list(head);          /* 10 -> 20 -> 30 -> NULL */
free_list(head);
head = NULL;
return 0;
}

```

3. Common Pitfalls & Bugs

- **Memory leak:** forgetting to `free()` every node when destroying a list — must walk node-by-node, saving the `next` pointer *before* freeing the current node (freeing then reading `cur->next` is use-after-free).
- **Dangling pointer after free:** freeing a node but leaving another pointer (e.g., `head`) still referencing it — always null out or reassign pointers immediately after freeing what they point to.
- **Losing the head:** overwriting `head` before saving a reference to the original first node, orphaning the entire rest of the list (unreachable = leaked).
- **Off-by-one in traversal:** checking `cur->next != NULL` instead of `cur != NULL` skips the last node or crashes when the list is empty.
- **Not checking `malloc`'s return** inside `create_node` — a NULL dereference under memory pressure.
- Confusing singly vs. doubly linked list pointer updates when inserting/deleting in the middle — forgetting to update **both** the `prev` and `next` links on both neighboring nodes in a doubly linked list.

4. Practical Code Example — full CRUD on a singly linked list (insert, delete, search):

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int data; struct Node *next; } Node;

Node* create_node(int v) {
    Node *n = malloc(sizeof(Node));
    if (!n) exit(1);
    n->data = v; n->next = NULL;
    return n;
}

void append(Node **head, int v) {
    Node *n = create_node(v);
    if (*head == NULL) { *head = n; return; }
    Node *cur = *head;
    while (cur->next != NULL) cur = cur->next; /* walk to tail: O(n)
for singly list w/o tail ptr */
    cur->next = n;
}

```

```

int delete_value(Node **head, int value) {
    Node *cur = *head, *prev = NULL;
    while (cur != NULL) {
        if (cur->data == value) {
            if (prev == NULL) *head = cur->next; /* deleting the head
node */
            else prev->next = cur->next;          /* bridge over the
deleted node */
            free(cur);
            return 1; /* success */
        }
        prev = cur;
        cur = cur->next;
    }
    return 0; /* not found */
}

void print_list(Node *head) {
    for (Node *c = head; c; c = c->next) printf("%d -> ", c->data);
    printf("NULL\n");
}

void free_list(Node *head) {
    while (head) { Node *next = head->next; free(head); head = next; }
}

int main(void) {
    Node *head = NULL;
    append(&head, 1); append(&head, 2); append(&head, 3); append(&head,
4);
    print_list(head);
    delete_value(&head, 2);
    print_list(head);
    free_list(head);
    return 0;
}

```

PROF

5. Hands-On Exercises

- *Basic:* Write an `int count_nodes(Node *head)` and `int sum_list(Node *head)` function; test on a list of 5 integers.
- *Challenging:* Implement iterative **in-place reversal** of a singly linked list (`Node* reverse(Node *head)`), $O(n)$ time, $O(1)$ extra space using three pointers `prev/cur/next`), then implement a **cycle-detection** function using Floyd's tortoise-and-hare algorithm, and demonstrate it correctly identifying a manually-constructed cyclic list (careful: do NOT try to `free()` a cyclic list without breaking the cycle first, or you'll infinite-loop/double-free).

XIII. File Management in C (*extended treatment*)

1. Concept Breakdown

File I/O in C is mediated through a **FILE*** — a handle to a buffered stream wrapping the OS's underlying file descriptor. Opening a file asks the OS to allocate kernel-side bookkeeping (position, permissions) and returns a handle; the C library layer adds its own userspace buffer on top to reduce the number of actual system calls (each **read/write** syscall is comparatively expensive). All file operations **MUST** be paired with **fclose()** to flush buffers and release both the OS descriptor and the library's internal structures — this is conceptually identical to **malloc/free** pairing, just for OS resources instead of heap memory.

Stack vs Heap relevance for file I/O:

Your FILE *fp variable position, fd) lives on the STACK internally (just a pointer, 8 bytes) flushes buffer	The FILE struct itself (buffer, is allocated on the HEAP by fopen() fclose() frees that heap structure &
fp -----> [internal buffer fd=3 position=142 ...]	

2. Syntax & Standard Usage

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* --- Writing --- */
    FILE *fp = fopen("data.txt", "w");    /* "w"=write (truncate),
    "a"=append, "r"=read,                  add "b" for binary mode
    e.g. "wb", "rb" */
    if (fp == NULL) {                      /* ALWAYS check – file may
    not be openable                          (permissions, disk
                                           full, path missing) */
        perror("fopen failed");
        return 1;
    }
    fprintf(fp, "%s,%d,%.2f\n", "Alice", 92, 3.8);
    fprintf(fp, "%s,%d,%.2f\n", "Bob", 85, 3.5);
    fclose(fp);                            /* flush + release –
    CRITICAL, do not skip */

    /* --- Reading line by line --- */
    fp = fopen("data.txt", "r");
    if (fp == NULL) { perror("fopen failed"); return 1; }
    char line[128];
    while (fgets(line, sizeof(line), fp) != NULL) {
        printf("Read: %s", line);
    }
}
```



```

    }
    fclose(fp);

    /* --- Binary I/O (struct records) --- */
    typedef struct { char name[20]; int score; } Record;
    Record r = {"Carol", 77};
    FILE *bf = fopen("records.bin", "wb");
    if (bf) { fwrite(&r, sizeof(Record), 1, bf); fclose(bf); }

    Record loaded;
    bf = fopen("records.bin", "rb");
    if (bf) {
        fread(&loaded, sizeof(Record), 1, bf);
        printf("Loaded: %s %d\n", loaded.name, loaded.score);
        fclose(bf);
    }

    return 0;
}

```

3. Common Pitfalls & Bugs

- **Not checking `fopen`'s return value** — a NULL `FILE*` used with `fprintf/fread` is undefined behavior/crash. Always check, and use `perror()` to report *why* (permissions, missing file, etc.).
- **Resource leak:** forgetting `fclose()` — on long-running programs (e.g., an embedded device logging continuously, relevant to your ESP32 attendance system's offline SPIFFS/LittleFS buffer) this exhausts the OS's file-descriptor limit over time.
- **Mode mismatch:** opening in `"w"` mode truncates/erases an existing file's contents immediately upon open — a very common "I just deleted my data" bug when the intent was `"a"` (append).
- **Text vs binary mode confusion:** on Windows, text mode silently translates `\n` ↔ `\r\n`; omitting `"b"` when working with raw binary struct data corrupts it on such platforms.
- **Partial `fread/fwrite`:** not checking the returned item count — a short read (e.g., truncated/corrupted file) silently leaves part of your buffer with stale/uninitialized data unless you check `fread(...) == expected_count`.
- **Buffering surprises:** writes may sit in the userspace buffer and not actually reach disk until `fclose()/fflush()` — a crash before that point can lose "written" data. Use `fflush(fp)` for critical durability points.

4. Practical Code Example — a small append-only log system (conceptually mirroring an offline attendance-log buffer):

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int log_event(const char *filename, const char *event) {
    FILE *fp = fopen(filename, "a");    /* append: never destroys prior
    log entries */

```

```

    if (fp == NULL) {
        perror("Could not open log file");
        return -1;
    }
    time_t now = time(NULL);
    fprintf(fp, "[%ld] %s\n", (long)now, event);
    fflush(fp); /* force to disk immediately:
durability for a log */
    fclose(fp);
    return 0;
}

int count_lines(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) return -1;
    int count = 0;
    char buf[256];
    while (fgets(buf, sizeof(buf), fp) != NULL) count++;
    fclose(fp);
    return count;
}

int main(void) {
    log_event("attendance.log", "Student 1023 punched IN");
    log_event("attendance.log", "Student 1024 punched IN");
    log_event("attendance.log", "Student 1023 punched OUT");
    printf("Total log entries: %d\n", count_lines("attendance.log"));
    return 0;
}

```

5. Hands-On Exercises

- *Basic*: Write a program that writes 5 student records (name, score) to a text CSV file, then reads them back and prints them, computing the average score from the file contents (not from memory).
- *Challenging*: Build a fixed-size binary record store mimicking a simplified offline attendance buffer: define a `struct AttendanceRecord { int student_id; long timestamp; char direction; }` (matching your project's IN/OUT dual-punch concept), write functions `append_record()`, `read_all_records()`, and `find_records_by_id(int id)` that seek through the binary file using `fseek/ftell` rather than loading the whole file into memory — and handle the case where the file doesn't exist yet (auto-create it) as well as a corrupted/partial trailing record (via checked `fread` counts).

Closing Notes: How to Use This Guide

- Compile everything with strict flags to catch real bugs early: `gcc -std=c11 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -g file.c -o app`
- Run heap-related exercises through **Valgrind** (`valgrind --leak-check=full ./app`) or **AddressSanitizer** — both directly relevant to your embedded/ESP32 work where memory bugs

are especially costly (no OS to catch a fault gracefully).

- For every "Challenging" exercise, try to first predict on paper what the memory (stack/heap) looks like at each step before running the code — this is the habit that separates people who *use* C from people who *understand* C.